A cryptographic extension of attribute-based communication and its implementation with a secure publish-subscribe architecture

XXXXXX XXXXXXXXXX
YYYYYYYYYYYYYYY, ZZZZZZ
wwwwww.zzzzzzz@institution.org

Attribute-based communication is a novel communication paradigm in which participants select their counterparts by using predicates over a set of attributes. This communication pattern forms the basis of the calculus AbC, which enables the specification and programming of a wide range of scenarios, from classical problems in distributed systems to IoT applications. However, in its original formulation, AbC does not guarantee security properties, such as integrity and confidentiality. In this paper, we propose that CRAbC (Cryptographic Attribute-based Calculus), an extension of AbC that integrates cryptographic primitives. Moreover, we propose a formal description of an attribute-based publish-subscribe infrastructure used to implement an execution environment for CRAbC.

1 Introduction

Attribute-based communication is a novel paradigm introduced in [10] to model *Collective Adaptive Systems* (CAS) [12]. The key feature of this paradigm is the possibility of dynamic selection of communication groups while accounting for runtime properties and the status of interacting entities.

Unlike classic approaches, participants select their counterparts in a communication by using predicates. To send a message v, one has to indicate the predicate π that must be satisfied by the possible receivers. Symmetrically, receivers can filter incoming messages using predicates on the received data and the sender's properties. Since these predicates are also parametrised with *local attributes* and when their values change, the interaction groups do implicitly change, allowing opportunistic interactions.

This paradigm was formalised in AbC [1, 6], a kernel calculus specifically thought to study the impact of attribute-based communication in the realm of CAS [7]. In AbC, a system consists of a multitude of components, each of which is equipped with a set of attributes describing their features, which can change at runtime. Component interactions are driven by conditions over their states to enable anonymity, adaptivity and open-endedness. AbC communication model follows broadcast in the style of [19]; output action can take place even without the presence of any receivers while input action instead waits to synchronise with available messages. The expressive power of AbC in terms of representing different communication paradigms, such as point-to-point, group-based, channel-based and broadcast-based models have been demonstrated in [2, 5].

A number of frameworks and infrastructures have been proposed to support runtime-execution of AbC systems. In [4, 3], the authors present a distributed coordination infrastructure for attribute-based interaction, providing a middleware layer that realises predicate-based partner selection in a scalable distributed setting. Another implementation is the one proposed in [15, 16], where a domain-specific framework, named ABEL, is introduced, that enables the practical programming of attribute-based systems while preserving the abstractions of the underlying calculus.

Submitted to:
ICE 2026

Attribute-based communication has been used in a wide range of scenarios, from classical problems in distributed systems to IoT applications. In particular, [17] introduces AbU, a calculus that integrates Event-Condition-Action rules with attribute-based interaction, providing a formal model for distributed event-driven programming in IoT scenarios. In [18], the formal verification of security and safety properties in distributed IoT ensembles is conducted by exploiting behavioural equivalences to reason about information flows and unintended interactions. AbU has also been used in [9] to investigate transactional coordination mechanisms for smart systems, showing how attribute-based interaction can be integrated with publish/subscribe infrastructures to support distributed IoT environments.

Unfortunately, in its original formulation, ABC does not guarantee security properties, such as integrity and confidentiality. To overcome these properties, in this paper, we propose CRABC (Cryptographic Attribute-based Calculus), an extension of ABC that integrates cryptographic primitives. Moreover, a variant of the classical publish-subscribe paradigm [11] is proposed that integrates attribute-based mechanisms together with a secure infrastructure that guarantees satisfaction of security properties. Finally, we show how the proposed framework can be used to implement an execution environment for CRABC.

2 CRABC: A Cryptographic Extension of ABC

In this Section, an extension of the Attribute-based Calculus ABC is presented to overcome the limitations outlined above. The proposed extension, named CRABC, integrates *cryptographic primitives* that guarantee data confidentiality and integrity, as well as authentication and non-repudiation.

Types and Values. Differently from ABC, which is agnostic with respect to the specific data types used in a specification, CRABC is a typed language. We let $\mathbb{T}$ be the set of data types that can be used in the language. Elements $\mathbb{T}$ will be denoted by τ, sometimes with pedices or apeces. We do not specify the exact content of $\mathbb{T}$; we will only assume that it contains standard data types like integers and booleans (denoted by int and bool) and type constructors like pair ($\_ * \_$). Other data types will be introduced in the paper to represent specific classes of values, such as *cryptographic keys* and *identities*.

We let $\mathbb{VAL}$ be the set of values that can be exchanged by communicating agents. Each $v \in \mathbb{VAL}$ is mapped to a type $\tau \in \mathbb{T}$. This mapping is defined in terms of the function $\mathsf{typeof} : \mathbb{VAL} \to \mathbb{T}$. In what follows, for each $\tau \in \mathbb{T}$, we will use $\mathbb{VAL}_\tau$ to denote $\{v \in \mathbb{VAL} \mid \mathsf{typeof}(v) = \tau\}$. For each $\tau \in \mathbb{T}$, we let $\bot_\tau \in \mathbb{VAL}_\tau$ denote an *unknown value* of type τ. For each value v and $\tau = \mathsf{typeof}(v)$, with an abuse of notation, we will use $\bot_v$ in place of $\bot_\tau$.

Given an enumerable set X, $\alpha : X \to \mathbb{T}$ and $\beta : X \to \mathbb{VAL}$, we say that α and β are *type coherent*, written $\alpha \triangleright \beta$, if and only if for any $x \in X$, $\mathsf{typeof}(\beta(x)) = \alpha(x)$.

Attributes. We let $\mathbb{A}$ be a set of attributes. These are identifiers that refer to relevant properties/features of components. An *attribute environment* is a function mapping attributes to values.

Keys and Cryptographic primitives. We let $\mathbb{K} \subset \mathbb{VAL}$ denote the set of *cryptographic keys* κ while $\mathsf{key} \in \mathbb{T}$ denotes the data type keys. Given a value $v \in \mathbb{VAL}$ and a key $\kappa \in \mathbb{K}$, we let $[: v :]_\kappa \in \mathbb{VAL}$ denote *ciphertext* obtained from the encryption of v with the key κ. For each $\tau \in \mathbb{T}$, $\mathsf{cipher}\ \tau$ denotes the data type of *ciphertext* containing values of type τ.

Given a $\kappa \in \mathbb{K}$ and $v \in \mathbb{VAL}$, function $enc(v, \kappa)$ is used to build $[: v :]_\kappa$. The symmetric operation *dec* can be used to obtain a *plaintext* from e *cyphertext* given a key. If the key used to decrypt the

e	::=	v	Value	π	::=	$\mathsf{a} \bowtie e$	Attribute test
	|	x	Variable		|	$\neg\pi$	Negation
	|	a	Attribute		|	$\pi_1 \wedge \pi_2$	Conjunction
	|	$\mathsf{this.a}$			|	$\pi_1 \vee \pi_2$	Disjunction
	|	e_1 *bop* e_2	Binary Operator				
	|	*uop* e_1	Unary Operator	where $\bowtie \in \{<, \leq, =, \geq, >\}$			
	|	$\mathsf{enc}(e_1, e_2)$	Encryption				
	|	$\mathsf{dec}(e_1, e_2)$	Decryption				
	|	$\mathsf{sign}(e)$	Signature				
	|	$\mathsf{whois}(e)$	Identity Checking				
	|	self	Self reference				

Figure 1: Syntax of Expressions and Predicates.

message *matches* the one used during the encryption, the secret value is revealed. If the decryption key is incorrect, an unknown value is returned. Given $\kappa_1, \kappa_2 \in \mathbb{K}$, we will use $\kappa_1 \leftrightarrow \kappa_2$ to denote that κ_1 matches with κ_2 and function *dec* can be formally defined as follows:

$$dec([: v :]_{\kappa_1}, \kappa_2) = \begin{cases} v & \kappa_1 \leftrightarrow \kappa_2 \\ \bot_v & \text{otherwise} \end{cases} \tag{1}$$

The use of predicate $\kappa_1 \leftrightarrow \kappa_2$ allows us to model the two main approaches to cryptography: *symmetric keys* and *public-private keys*. When a *symmetric encryption* is used, $\leftrightarrow$ corresponds to the identity predicate. Indeed, in this case, to obtain the content of a ciphertext, the same key used during the encryption must be used. On the other hand, in *asymmetric approach* a pair of keys is used: the *secret key* κ^S, known only to a single agent, and its *public* counterpart κ^P, such that $\kappa^S \leftrightarrow \kappa^P$ and $\kappa^P \leftrightarrow \kappa^S$. By using the asymmetric approach, we have that $[: v :]_{\kappa^P}$ can be received only by the ones that know κ^S, while $[: v :]_{\kappa^S}$ is the proof that v comes from someone knowing the secret key. The secret key can be known to single participants, like in RSA [20], or to specific groups of agents.

Identities. One of the aspects that differentiates CRABC from ABC is the fact that each component is associated with a *unique* identity. We let $\mathbb{I} \subseteq \mathbb{VAL}$ denote the set of *identities* ι having type $\mathsf{id} \in \mathbb{T}$. Every value ι can be used to sign messages. We let $\{: v :\}_\iota$ denote the value v signed by ι. We also let $\mathsf{sign}\ \tau \in \mathbb{T}$ to denote the signed values of type $\tau \in \mathbb{T}$. Given $\iota \in \mathbb{I}$ and $v \in \mathbb{VAL}$ we let $\mathsf{attest}(v, \iota)$ denote the used to compute $\{: v :\}_\iota$. Conversely, function $\mathsf{whois}(\{: v :\}_\iota)$ yields the identity ι of the participant that signed the value v.

Expressions. Syntax of expressions e is reported in Figure 1. Expressions are built from values $v \in \mathbb{VAL}$, variables $\mathsf{x} \in \mathbb{VAR}$ and attributes $\mathsf{a} \in \mathbb{A}$ by using standard binary/unary operators. For the sake of generality, we do not specify here the set of *binary and unary operators*, denoted in the syntax with $\cdot$ *bop* $\cdot$ and *uop* $\cdot$, respectively. We only assume that each operator *bop* (resp. *uop*) is associated with a function $f_{bop} : \tau_1 \times \tau_2 \to \tau$ (resp. $f_{uop} : \tau_1 \to \tau$) mapping a pair of type $\tau_1 * \tau_2$ (resp. a value of type τ_1) with a value of type τ. Expression can also contain cryptographic primitives that can be used to *encrypt or decrypt values*, to sign a value with the identity of the component that evaluates the expression, and to extract the identity of the component that signed a given value. Finally, the expression self is used to refer to the identity of the component where the expression is evaluated.

Figure 2 contains the inference rules used to derive the type of an expression e. These derivations depend on an *attributes type environment* $\Lambda_\mathbb{A}$ and on a *variables type environment* $\Lambda_\mathbb{V}$. These are partial

$$\frac{\mathsf{typeof}(v)}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash v : \tau} \qquad \frac{\Lambda_{\mathbb{V}}(\mathtt{x}) = \tau}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathtt{x} : \tau} \qquad \frac{\Lambda_{\mathbb{A}}(\mathsf{a}) = \tau}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{a} : \tau} \qquad \frac{\Lambda_{\mathbb{A}}(\mathsf{a}) = \tau}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{this.a} : \tau}$$

$$\frac{f_{\,bop} \ : \tau_1 * \tau_2 \to \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_1 : \tau_1 \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_2 : \tau_2}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_1 \ bop \ e_2 : \tau}$$

$$\frac{f_{uop} \ : \tau_1 \to \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_1 : \tau_1}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash uop \ e_1 : \tau} \qquad \frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_1 : \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_2 : \mathsf{key}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{enc}(e_1,e_2) : \mathsf{cipher}\ \tau}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_1 : \mathsf{cipher}\ \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_2 : \mathsf{key}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{dec}(e_1,e_2) : \tau}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \tau}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{sign}(e) : \mathsf{sign}\ \tau} \qquad \frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \mathsf{sign}\ \tau}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{whois}(e) : \mathsf{id}} \qquad \frac{}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{self} : \mathsf{id}}$$

Figure 2: Typing rules for Expressions.

$$[\![v]\!]^{\iota}_{\Gamma} = v \qquad [\![\mathsf{a}]\!]^{\iota}_{\Gamma} = \Gamma(\mathsf{a}) \qquad [\![\mathsf{this.a}]\!]^{\iota}_{\Gamma} = \Gamma(a)$$

$$\frac{[\![e_1]\!]^{\iota}_{\Gamma} = v \quad [\![e_2]\!]^{\iota}_{\Gamma} = \kappa}{[\![\mathsf{enc}(e_1,e_2)]\!]^{\iota}_{\Gamma} = [\!: v :]_{\kappa}} \qquad \frac{[\![e_1]\!]^{\iota}_{\Gamma} = [\!: v :]_{\kappa} \quad [\![e_2]\!]^{\iota}_{\Gamma} = \kappa'}{[\![\mathsf{dec}(e_1,e_2)]\!]^{\iota}_{\Gamma} = dec([\!: v :]_{\kappa}, \kappa')}$$

$$\frac{[\![e_1]\!]^{\iota}_{\Gamma} = v_1 \quad [\![e_2]\!]^{\iota}_{\Gamma} = v_2}{[\![e_1 \ bop \ e_2]\!]^{\iota}_{\Gamma} = f_{bop}(v_1,v_2)} \qquad \frac{[\![e_1]\!]^{\iota}_{\Gamma} = v_1}{[\![uop \ e_1]\!]^{\iota}_{\Gamma} = f_{uop}(v_1)}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = v}{[\![\mathsf{sign}(e)]\!]^{\iota}_{\Gamma} = \{\!: v :\}_{\iota}} \qquad \frac{[\![e]\!]^{\iota}_{\Gamma} = \{\!: v :\}_{\iota'}}{[\![\mathsf{whois}(e)]\!]^{\iota}_{\Gamma} = \iota'} \qquad [\![\mathsf{self}]\!]^{\iota}_{\Gamma} = \iota$$

Figure 3: Expression evaluation function.

functions mapping attributes and variables to types, respectively. We say that an expression e is *well-formed* in $\Lambda_{\mathbb{A}}$ and $\Lambda_{\mathbb{V}}$ if and only if there exists a type τ such that $\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \tau$.

We say that an expression e is *ground* if and only if it does not contain any variable. Note that, if e is ground, it can be typed only according to a type attribute environment $\Lambda_{\mathbb{A}}$. A *ground* expression e can be evaluated according to an *attribute environment* Γ and an identity value ι. The former is a function that maps attributes to values, whereas the latter is the identity function, in which the expression is evaluated. The evaluation function $[\![e]\!]^{\iota}_{\Gamma}$ is defined in Figure 3. The following lemma guarantees that if e is ground and τ-typed under $\Lambda_{\mathbb{A}}$, then $[\![v]\!]^{\iota}_{\Gamma}$ results in a value of type τ whenever $\Lambda_{\mathbb{A}} \triangleright \Gamma$.

Lemma 1 *For any ground expression e,* attribute type environment $\Lambda_{\mathbb{A}}$ *and* environment Γ*, such that* $\Lambda_{\mathbb{A}} \triangleright \Gamma$*,* $\Lambda_{\mathbb{A}} \vdash e : \tau$ *if and only if* $[\![e]\!]^{\iota}_{\Gamma} = v$ *and* $\mathsf{typeof}(v) = \tau$.

Evaluation of an expression under an attribute environment Γ and an identity ι, yields a value v obtained by replacing all the attributes in e with the corresponding values in Γ. When attribute-based communication is used, an agent may send other participants expressions that are only partially evaluated. We let the *closure* of e under Γ and ι, $e \Downarrow^{\iota}_{\Gamma}$, as defined in Figure 4, where:

$$v \Downarrow^{\iota}_{\Gamma} = v \qquad \mathsf{a} \Downarrow^{\iota}_{\Gamma} = \mathsf{a} \qquad \mathsf{this.a} \Downarrow^{\iota}_{\Gamma} = \Gamma(\mathsf{a})$$

$$\frac{[\![e_1]\!]^{\iota}_{\Gamma} = v \quad [\![e_2]\!]^{\iota}_{\Gamma} = \kappa}{\mathsf{enc}(e_1, e_2) \Downarrow^{\iota}_{\Gamma} = [: v :]_{\kappa}} \qquad \frac{[\![e_1]\!]^{\iota}_{\Gamma} = [: v :]_{\kappa} \quad [\![e_2]\!]^{\iota}_{\Gamma} = \kappa'}{\mathsf{dec}(e_1, e_2) \Downarrow^{\iota}_{\Gamma} = dec([: v :]_{\kappa}, \kappa')}$$

$$\frac{e_1 \Downarrow^{\iota}_{\Gamma} = e_1' \quad e_2 \Downarrow^{\iota}_{\Gamma} = e_2'}{e_1 \; bop \; e_2 \Downarrow^{\iota}_{\Gamma} = apply(e_1' \; bop \; e_2')} \qquad \frac{e_1 \Downarrow^{\iota}_{\Gamma} = e_1'}{uop \; e_1 \Downarrow^{\iota}_{\Gamma} = apply(uop \; e_1')}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = v}{\mathsf{sign}(e) \Downarrow^{\iota}_{\Gamma} = \{: v :\}_{\iota}} \qquad \frac{[\![e]\!]^{\iota}_{\Gamma} = \{: v :\}_{\iota'}}{\mathsf{whois}(e) \Downarrow^{\iota}_{\Gamma} = \iota'} \qquad \mathsf{self} \Downarrow^{\iota}_{\Gamma} = \iota$$

Figure 4: Closure of Expressions

$$apply(e_1 \; bop \; e_2) = \begin{cases} f_{bop}(v_1, v_2) & e_1 = v_1 \wedge e_2 = v_2 \\ e_1 \; bop \; e_2 & \text{otherwise} \end{cases}$$

$$apply(uop \; e_1) = \begin{cases} f_{uop}(v_1) & e_1 = v_1 \\ uop \; e_1 & \text{otherwise} \end{cases}$$

The following lemma guarantees that, like for evaluation, the closure of an expression preserves its type.

Lemma 2 *For any ground expression e,* attribute type environment $\Lambda_{\mathbb{A}}$ *and* environment Γ*, such that* $\Lambda_{\mathbb{A}} \triangleright \Gamma$*,* $\Lambda_{\mathbb{A}} \vdash e : \tau$ *if and only if* $e \Downarrow^{\iota}_{\Gamma} = e'$ *and* $\Lambda_{\mathbb{A}} \vdash e' : \tau$*.*

Processes. The syntax of CRABC processes is the same as the one defined for ABC and it is defined as follows:

$$\begin{array}{rcl} P, Q & ::= & \mathsf{skip} \\ & | & \langle e \rangle @ \pi ; P \\ & | & \pi(\mathtt{x} : \tau) \Rightarrow P \\ & | & \mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q \\ & | & \mathsf{when}(e)\ P \\ & | & [\tilde{\mathtt{a}} \leftarrow \tilde{e}]; P \\ & | & P \mid Q \\ & | & \mathsf{N}[\tilde{e}] \end{array}$$

Process skip represents the process that does not perform any action, while $\langle e \rangle @ \pi ; P$ is a process that sends the evaluation of e to all components satisfying predicate π after closure. A predicate, whose syntax is defined in Figure 1, consists of a boolean formula on attributes. The input prefix $\pi(\mathtt{x} : \tau) \Rightarrow P$ is used to receive a message of type τ from a sender that satisfies the predicate π. When the value of v is received, process $P[\mathtt{x} \leftarrow v]$, namely the process where x is replaced by v, is activated. Input prefix acts as a binder for the variable x. Given a process P, we say that a variable x is bound in P if and only if it occurs under the scope of an input action on x. As usual, we say that P is *closed* if and only if all the variables occurring in P are *bound*. Conditionals if (e) then P else Q behave as P or Q depending on the truth value of guard while $\mathsf{when}(e)\ P$ is the process that behaves like P when condition e is satisfied. The update prefix $[\tilde{a} \leftarrow \tilde{e}]; P$ is used to update the values of attributes $\tilde{\mathtt{a}}$ with the evaluation of expressions $\tilde{e}$, where

$$\frac{}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{skip}} \qquad \frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \pi : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \langle e \rangle @ \pi ; P}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}}[\mathtt{x} \leftarrow \tau] \vdash \pi : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}}[\mathtt{x} \leftarrow \tau] \vdash P}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \pi(\mathtt{x} : \tau) \Rightarrow P}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash Q}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{when}(e)\ P}$$

$$\frac{\forall i, \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_i : \Lambda_{\mathbb{A}}(\mathtt{a}_i) \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash [\mathtt{a}_1,\ldots,\mathtt{a}_k \leftarrow e_1,\ldots,e_k];P}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash Q}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P \mid Q}$$

$$\frac{\mathsf{N}(\mathtt{x}_1 : \tau_1,\ldots,\mathtt{x}_k : \tau_k) \stackrel{\triangle}{=} P \in \Delta \quad \forall i, \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash e_i : \tau_i}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{N}[e_1,\ldots,e_k]}$$

Figure 5: Process typing rules.

we use $\tilde{\cdot}$ to denote a non-empty sequence of terms. Process $P \mid Q$ represents the parallel composition of P and Q. Finally, a *process call* $\mathsf{N}[\tilde{e}]$ is used to implement recursive behaviours. We assume a set of *agent definitions* Δ of the form $\mathsf{N}(x\,\tilde{:}\,\tau) \stackrel{\triangle}{=} P$ associating each agent name N with a process P. For each $\mathsf{N}(x\,\tilde{:}\,\tau) \stackrel{\triangle}{=} P \in \Delta$, any process call occurs under an action prefix operator in P. To check if a process is using attributes and expressions with the correct type, inference rules in Figure 5 are introduced. We say that a process is P *well-formed* if and only if $\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P$ can be proved. Moreover, a set of process definitions Δ is well-formed, written $\Lambda_{\mathbb{A}} \vdash \Delta$, if and only if for each $\mathsf{N}(x\,\tilde{:}\,\tau) \stackrel{\triangle}{=} P \in \Delta$, P is well-formed.

Components, Configurations and System Specifications. Like in ABC, a CRABC system consists of a set of *components* that interact with each other. However, while in ABC a component is a process running under an attribute environment, in CRABC each component is equipped also with a unique identifier ι and has the form $[\Gamma \triangleright P]_\iota$. A configuration is then obtained as the parallel composition of components. Formally, the syntax for components is defined as follows:

$$C ::= [\Gamma \triangleright P]_\iota \mid C_1 || C_2$$

A *system specification* consists of a configuration that is evaluated under: an *attribute type environment* $\Lambda_{\mathbb{A}}$ containing the declarations of all the attributes used by the components; a *process definition* Δ.A system specification S has form:

$$\Lambda_{\mathbb{A}}, \Delta \triangleright C$$

Finally, we say that a system specification $S = \Lambda_{\mathbb{A}}, \Delta \triangleright C$ is well-formed, if and only if Δ is well-formed ($\Lambda_{\mathbb{A}} \vdash \Delta$) and the configuration C is well-formed according to $\Lambda_{\mathbb{A}}$ and Δ:

$$\frac{\forall \mathtt{a}, \mathsf{typeof}(\Gamma(\mathtt{a})) = \Lambda_{\mathbb{A}}(\mathtt{a}) \quad \Lambda_{\mathbb{A}},\emptyset \vdash P}{\Lambda_{\mathbb{A}}, \Delta \triangleright [\Gamma \triangleright P]_\iota} \qquad \frac{\Lambda_{\mathbb{A}}, \Delta \triangleright C_1 \quad \Lambda_{\mathbb{A}}, \Delta \triangleright C_2}{\Lambda_{\mathbb{A}}, \Delta \triangleright C_1 || C_2}$$

2.1 Operational semantics

The operational semantics of CRABC follows the two-level structure already adopted for ABC, separating the evolution of processes inside a component from the evolution of whole system configurations.

Process-level At the first level, we define a transition relation describing how a process evolves when executed within a component. Transitions are written as

$$P \xrightarrow[\iota:\Gamma]{\ell} P',$$

meaning that process P, running inside the component identified by ι and with attribute environment Γ, performs an action ℓ and evolves to P'. Component-level transitions are labelled to record the interaction between a process and its execution context. The set of process-level labels is defined as follows.

$$\ell ::= \Gamma \triangleright \overline{\pi(v)} \mid \Gamma \triangleright \pi(v) \mid [\tilde{a} \leftarrow \tilde{v}].$$

Labels are used to denote communication and local evolution. An output action $\Gamma \triangleright \overline{\pi(v)}$ represents the sending of a value v under predicate π. An input action $\Gamma \triangleright \pi(v)$ denotes the reception of a value when the predicate π is satisfied. Finally, an update action $[a \tilde{\leftarrow} \tilde{v}]$ captures a local modification of the attribute environment.

An auxiliary relation is needed to model the fact that a process is not able to receive a message. This is rendered in terms of a relation of the form:

$$P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}$$

indicating that P, running at component ι with attribute environment Γ_1, is not able to receive value v sent to the components satisfying π by a component with attribute environment Γ_2. Both the transition relations $\dot{\rightarrow}$ and $\dot{\leftarrow}$ are defined by the rules in Figure 6 and 7.

The inactive process skip does not perform any action and can only discard incoming messages (NIL-SKIP).

The term $\langle e \rangle @ \pi; P$ performs an output action and continues as P (SEND). As in ABC, the output is non-blocking and does not require the presence of receivers. Since an output prefix does not enable reception, any attempted input is discarded and propagated to the continuation (SEND-SKIP). The term $\pi(\tilde{x}) \Rightarrow P$ performs an input action when the sender and receiver predicates are mutually satisfied (RECV). In this case, the process evolves to $P[\tilde{x} \leftarrow \tilde{v}]$. Otherwise, the message is discarded (RECV-SKIP). This dual compatibility check characterises the attribute-based nature of communication.

A conditional $\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q$ behaves as the selected branch, propagating its actions accordingly(IF-T, IF-F). Discarding follows the active branch (IFT-SKIP, IFF-SKIP). The awareness construct $\mathsf{when}(e)\ P$ behaves as P when the guard holds (WHEN). When the guard is false, it stutters and discards incoming messages (WHEN-SKIP-F); when the guard is true, discarding is inherited from P (WHEN-SKIP-T).

Parallel composition $P \mid Q$ lifts actions from either component (PAR-L, PAR-R). A message is discarded only when both components discard it (PAR-SKIP).

A process of the form $[\tilde{a} \leftarrow \tilde{e}]; P$ performs a local update action and continues as P (UPDATE). If an incoming message cannot be received, it is discarded (UPDATE-SKIP).

The term $\mathsf{N}[\tilde{e}]$ unfolds according to its defining equation in Δ and inherits the transition behaviour of the instantiated body (REC). If the unfolded body discards an incoming message, the process call discards it as well (REC-SKIP).

$$\mathsf{skip} \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}} \quad \textsc{Nil-Skip}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = v \quad \pi \Downarrow^{\iota}_{\Gamma} = \pi'}{\langle e \rangle @ \pi ; P \xrightarrow[\iota:\Gamma_1]{\Gamma_2 \triangleright \overline{\pi'(v)}} P} \textsc{Send} \qquad \frac{P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}}{\langle e \rangle @ \pi ; P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{Send-Skip}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{true} \quad P \xrightarrow[\iota:\Gamma]{\ell} P'}{\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q \xrightarrow[\iota:\Gamma]{\ell} P'} \textsc{If-T} \qquad \frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{false} \quad Q \xrightarrow[\iota:\Gamma]{\ell} Q'}{\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q \xrightarrow[\iota:\Gamma]{\ell} Q'} \textsc{If-F}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{true}}{\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{IfT-Skip} \qquad \frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{false}}{\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{IfF-Skip}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{true} \quad P \xrightarrow[\iota:\Gamma]{\ell} P'}{\mathsf{when}(e)\ P \xrightarrow[\iota:\Gamma]{\ell} P'} \textsc{When}$$

$$\frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{true} \quad P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}}{\mathsf{when}(e)\ P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{When-Skip-T} \qquad \frac{[\![e]\!]^{\iota}_{\Gamma} = \mathsf{false}}{\mathsf{when}(e)\ P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{When-Skip-F}$$

Figure 6: Process-level operational semantics (Part 1)

$$\frac{P \xrightarrow[\iota:\Gamma]{\ell} P'}{P \mid Q \xrightarrow[\iota:\Gamma]{\ell} P' \mid Q} \textsc{Par-L} \qquad \frac{Q \xrightarrow[\iota:\Gamma]{\ell} Q'}{P \mid Q \xrightarrow[\iota:\Gamma]{\ell} P \mid Q'} \textsc{Par-R} \qquad \frac{P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}} \wedge Q \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}}{P \mid Q \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{Par-Skip}$$

$$\frac{[\![\tilde{e}]\!]^{\iota}_{\Gamma} = \tilde{v}}{[\tilde{a} \leftarrow \tilde{e}]; P \xrightarrow[\iota:\Gamma]{[\tilde{a} \leftarrow \tilde{v}]} P} \textsc{Update} \qquad \frac{P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}}{[\tilde{a} \leftarrow \tilde{e}]; P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi(v)}}} \textsc{Update-Skip}$$

$$\frac{\Gamma \models \pi' \quad \Gamma \models \pi[\tilde{x} \leftarrow \tilde{v}] \Downarrow^{\iota}_{\Gamma}}{\pi(\tilde{x}) \Rightarrow P \xrightarrow[\iota:\Gamma_1]{\Gamma_2 \triangleright \pi'(\tilde{v})} P[\tilde{x} \leftarrow \tilde{v}]} \textsc{Recv} \qquad \frac{\Gamma \not\models \pi' \ \vee \ \Gamma \not\models \pi[\tilde{x} \leftarrow \tilde{v}] \Downarrow^{\iota}_{\Gamma}}{\pi(\tilde{x}) \Rightarrow P \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi'(\tilde{v})}}} \textsc{Recv-Skip}$$

$$\frac{\mathsf{N}(\tilde{\mathrm{x}} : \tau) \stackrel{\triangle}{=} P \in \Delta \quad P[\tilde{\mathrm{x}} \leftarrow [\![\tilde{e}]\!]^{\iota}_{\Gamma}] \xrightarrow[\iota:\Gamma]{\ell} P'}{\mathsf{N}[\tilde{e}] \xrightarrow[\iota:\Gamma]{\ell} P'} \textsc{Rec}$$

$$\frac{\mathsf{N}(\tilde{\mathrm{x}} : \tau) \stackrel{\triangle}{=} P \in \Delta \quad P[\tilde{\mathrm{x}} \leftarrow [\![\tilde{e}]\!]^{\iota}_{\Gamma}] \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi'(\tilde{v})}}}{\mathsf{N}[\tilde{e}] \xleftarrow[\iota:\Gamma_1]{\widetilde{\Gamma_2 \triangleright \pi'(\tilde{v})}}} \textsc{Rec-Skip}$$

Figure 7: Process-level operational semantics (Part 2)

$$\frac{P \xrightarrow[\iota:\Gamma]{\Gamma \triangleright \overline{\pi(v)}} P'}{[\Gamma \triangleright P]_\iota \xRightarrow{\Gamma \triangleright \overline{\pi(v)}} [\Gamma \triangleright P']_\iota}\ \text{C-OUT} \qquad \frac{P \xrightarrow[\iota:\Gamma]{\Gamma \triangleright \pi(v)} P'}{[\Gamma \triangleright P]_\iota \xRightarrow{\Gamma \triangleright \pi(v)} [\Gamma \triangleright P']_\iota}\ \text{C-IN}$$

$$\frac{P \xrightarrow[\iota:\Gamma]{\widetilde{\Gamma \triangleright \pi(v)}} P}{[\Gamma \triangleright P]_\iota \xRightarrow{\tau} [\Gamma \triangleright P]_\iota}\ \text{C-SKIP} \qquad \frac{P \xrightarrow[\iota:\Gamma]{[\tilde{a} \leftarrow \tilde{v}]} P'}{[\Gamma \triangleright P]_\iota \xRightarrow{\tau} [\Gamma[\tilde{a} \leftarrow \tilde{v}] \triangleright P']_\iota}\ \text{C-UPD}$$

$$\frac{C_1 \xRightarrow{\tau} C_1'}{C_1 || C_2 \xRightarrow{\tau} C_1' || C_2}\ \text{PAR-}\tau\text{-L} \qquad \frac{C_2 \xRightarrow{\tau} C_2'}{C_1 || C_2 \xRightarrow{\tau} C_1 || C_2'}\ \text{PAR-}\tau\text{-R}$$

$$\frac{C_1 \xRightarrow{\Gamma \triangleright \overline{\pi(v)}} C_1' \quad C_2 \xRightarrow{\Gamma \triangleright \pi(v)} C_2'}{C_1 || C_2 \xRightarrow{\Gamma \triangleright \overline{\pi(v)}} C_1' || C_2'}\ \text{COMM-LR}$$

$$\frac{C_2 \xRightarrow{\Gamma \triangleright \overline{\pi(v)}} C_2' \quad C_1 \xRightarrow{\Gamma \triangleright \pi(v)} C_1'}{C_1 || C_2 \xRightarrow{\Gamma \triangleright \overline{\pi(v)}} C_1' || C_2'}\ \text{COMM-RL}$$

$$\frac{C_1 \xRightarrow{\Gamma \triangleright \pi(v)} C_1' \quad C_2 \xRightarrow{\Gamma \triangleright \pi(v)} C_2'}{C_1 || C_2 \xRightarrow{\Gamma \triangleright \pi(v)} C_1' || C_2'}\ \text{MULTI-IN}$$

Figure 8: Operational semantics of configurations

Lemma 3 *For each closed process P and Q and attribute type environment $\Lambda_{\mathbb{A}}$, attribute environments Γ such that $\Lambda_{\mathbb{A}} \triangleright \Gamma$, if $\Lambda_{\mathbb{A}}, \emptyset \vdash P$ and $P \xrightarrow[\iota:\Gamma]{\ell} Q$ then $\Lambda_{\mathbb{A}}, \emptyset \vdash Q$.*

Components-level At the configuration level, we describe the observable evolution of systems. Configuration transitions are written as

$$C \xRightarrow{\alpha} C'$$

Internal phenomena such as local updates or discarded messages are labelled as τ-transitions and defined as follows:

$$\alpha ::= \tau \mid \Gamma \triangleright \pi(v) \mid \Gamma \triangleright \overline{\pi(v)}.$$

Lemma 4 *For each configuration C_1 and C_2 and attribute type environment $\Lambda_{\mathbb{A}}$ and process definitions Δ, if $\Lambda_{\mathbb{A}}, \Delta \vdash C_1$ and $C_1 \xRightarrow{\alpha} C_2$ then $\Lambda_{\mathbb{A}}, \Delta \vdash C_2$.*

3 Secure Publish/Subscribe

In this section a publish/subscribe (pub/sub) architecture, named SECPUBSUB, that guarantees secure interactions between the involved participants is presented. The pub/sub paradigm is a communication paradigm in which producers of data (*publishers*) send messages to a shared communication channel without specifying the recipients. These messages are collected by *subscribers* that filter only the piece

of information they are interested in. Filtering of messages can be either *topic-based* or *content-based*. In the first case, messages are equipped with a *topic* at the time of publication. Subscribers register for a set of topics of the message they want to receive. In the *content-based* approach, subscribers indicate the properties about the content of the messages they are interested in, or their structure. Interaction between publishers and subscribers can be mediated by a *broker* that receives and distributes messages based on topics or content filters.

Unlike the standard pub/sub paradigm, in SECPUBSUB messages are filtered not only by predicates on their content, but also by the *attributes* of the publisher. Moreover, the latter can select the class of subscribers that can receive the message by using predicates on their attributes. Indeed, in SECPUBSUB an attribute-based pub/sub paradigm is adopted: all participants are equipped with a set of attributes used to control interactions.

The architecture of SECPUBSUB consists of: a *Certification Authority*, a *Broker*, and a set of *Agents*. To guarantee integrity, authentication, and non-repudiation, these components interact using a protocol in which messages are encrypted using asymmetric cryptography.

Certification Authority. In SECPUBSUB the *Certification Authority*, denoted by CA, has the responsibility of controlling the identities of the agents and regulating the access to predicates. We let κ^P_{CA} be the public key associated with the certification authority CA. The usage of this key guarantees that only the CA can extract info from the exchanged messages. Moreover, CA also plays the role of *identity provider* for the agents that interact in the system. For each identity ι, CA is able to retrieve the public key κ^P_ι. We let Reg be the function that, given an identity ι, returns its public key κ^P_ι if this exists, and $\perp$ otherwise.

For simplicity, in this paper, we do not consider all aspects of key distribution and identity registration that can be managed either at system configuration time or via standard protocols [14].

Finally, CA provides the machinery to implement access control on attributes. In other words, CA controls the usage of attributes, their visibility and update. This approach guarantees that an external observer cannot determine either the names of the attributes used or their values. Moreover, CA integrates the access control policies Π described in the previous section.

For the sake of simplicity, we model CA as an object exposing three methods: subscribe, publish and set. Method subscribe is used to validate the subscription of an agent located at ι:

$$\mathsf{CA.subscribe}\left(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota}\right)=\begin{cases}[\!:\iota_1,\tau,\pi:]_{\kappa^P_{\mathsf{CA}}} & \mathsf{Reg}(\iota)=\kappa^P_\iota\leftrightarrow\kappa^S_\iota\\ \perp & \text{otherwise}\end{cases}$$

The parameter of method subscribe is a message signed with the secret key of ι and containing the type τ of subscribed messages and the predicate π that the publisher must satisfy. This message is encrypted with the public key of CA. If the message is well-formed, i.e. the key used to sign the message belongs to ι (Reg is the registry used by CA to store public keys of identities), the method returns the pair τ,π encoded with the public key of CA. We will see later that the broker will store this pair for message delivery.

The CA is also responsible for checking if a message can be delivered or not to a specific agent. This is done by the method publish defined below:

$$\mathsf{CA.publish}\left(\{\!:[\!:v,\pi_1:]_{\kappa^P_{\mathsf{CA}}},\iota_1:\}_{\kappa^S_{\iota_1}},\Gamma_{\iota_1},[\!:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}},\iota_2,\Gamma_{\iota_2}\right)=$$
$$\begin{cases}[\!:v:]_{\kappa^P_{\iota_2}} & \Gamma_1\models\pi_2\wedge\Gamma_2\models\pi_1\wedge\tau=\mathsf{typeof}(v)\\ & \mathsf{Reg}(\iota_1)=\kappa^P_{\iota_1}\leftrightarrow\kappa^S_{\iota_1}\\ \perp & \text{otherwise}\end{cases}$$

Method publish is used to check if a given message v sent from ι_1 with attributes Γ_1 should be delivered at ι_2 with store Γ_2. In this case, CA checks that the message is of the expected type, and that the store of the two participants satisfies the requested predicates.

We can observe that the broker stores the attributes encrypted. This guarantees that an external observer cannot access the attribute values. For this reason, CA has also the responsibility to manage how attributes are stored in the broker. This is done by the method set defined below:

$$\mathsf{CA.set}\left(\{\!:[\!:\tilde{\mathsf{a}},\tilde{v}:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota}\right)=\begin{cases}[\!:\tilde{\mathsf{a}}:]_{\kappa^P_{\mathsf{CA}}},[\!:\tilde{v}:]_{\kappa^P_{\mathsf{CA}}} & \mathsf{Reg}(\iota)=\kappa^P_\iota\leftrightarrow\kappa^S_\iota\\ \bot & \text{otherwise}\end{cases}$$

When this method is invoked, if the component's identity is verified, the certification authority will provide the broker with ciphertexts containing the encrypted versions of the attributes and their values, which the broker will store.

Broker The *broker*, denoted by B, is the entity that mediates all interactions between publishers and subscribers: it receives publications, stores subscriptions, evaluates predicates, and delivers notifications. We can model it as the triple:

$$\mathsf{B}=[\Sigma,\mathsf{H},\mathsf{D}]$$

where $\Sigma:\mathbb{I}\to\mathsf{Cypher}(\mathbb{A})\to\mathsf{Cypher}(\mathbb{VAL})$ is the global attribute environment that associates each identifiers to a function mapping *ciphertexts* containing attributes to *ciphertexts* containing values, H is the set of registered handlers (subscriptions), and D is the dispatcher that holds the queues of outgoing messages.

A broker can receive the following messages:

$$\mathsf{set}(\{\!:[\!:\tilde{\mathsf{a}},\tilde{\mathsf{v}}:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota})\qquad \mathsf{pub}(\{\!:[\!:v,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota})$$

$$\mathsf{sub}(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})\quad \mathsf{sub}*(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})$$

Message $\mathsf{set}([\!:\tilde{\mathsf{a}}:]_{\kappa^P_{\mathsf{CA}}},[\!:\tilde{\mathsf{v}}:]_{\kappa^P_{\mathsf{CA}}})$ is sent by ι to communicate that attributes $\tilde{\mathsf{a}}$ are set to the values $\tilde{v}$. To guarantee both authentication and secrecy of the attribute values, this message is signed with ι's public key and encrypted with CA's public key.

A publication message $\mathsf{pub}(\{\!:[\!:v,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota})$ consists of a value v, a predicate π, and a publisher identity ι. As we have already seen, this message is signed with the publisher's private key and encrypted with *CA*'s public key. A subscription request $\mathsf{sub}(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})$ is used to communicate the interest in future publications matching the predicate π and compatible with the expected type τ. In the message, the handler h is the address of the callback endpoint where the received message will be delivered. Message $\mathsf{sub}*(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})$ is similar to the previous one; however, in this case the subscription is *persistent* and multiple messages can be received.

We are now ready to show how the broker reacts when these messages are received. The following two rules describe how a broker evolves when subscribing messages are received:

$$\frac{\mathsf{CA.subscribe}\left(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota}\right)=[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}}}{\mathsf{CA}\triangleright[\Sigma,\mathsf{H},\mathsf{D}]\xrightarrow{\mathsf{sub}(\{:[:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})}\mathsf{CA}\triangleright[\Sigma,(\iota,h,[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}})::\mathsf{H},\mathsf{D}]}$$

$$\frac{\mathsf{CA.subscribe}\left(\{\!:[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota}\right)=[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}}}{\mathsf{CA}\triangleright[\Gamma,\mathsf{H},\mathsf{D}]\xrightarrow{\mathsf{sub}*(\{:[:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})}\mathsf{CA}\triangleright[\Gamma,(\iota,*h,[\!:\tau,\pi:]_{\kappa^P_{\mathsf{CA}}})::\mathsf{H},\mathsf{D}]}$$

$$
\begin{array}{l}
\mathsf{let\ dispatch}(\mathsf{CA},\Sigma,\Gamma_{\iota_1},\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\mathsf{H}) = \\
\quad \mathsf{match\ H\ with} \\
\quad \mid\ [\,] \Rightarrow [\,],[\,] \\
\quad \mid\ (\iota_2,{*h},[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}}) :: \mathsf{H}_1 \Rightarrow \\
\qquad \mathsf{let\ H'},\mathsf{D'} = \mathsf{dispatch}(\mathsf{CA},\Sigma,\Gamma_{\iota_1},\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\mathsf{H}_1) \\
\qquad \mathsf{in} \\
\qquad\quad \mathsf{match} \\
\qquad\quad\ \mathsf{CA.publish}\left(\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\Gamma_{\iota_1},[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}},\iota_2,\Sigma(\iota_2)\right) \\
\qquad\quad \mathsf{with} \\
\qquad\qquad \mid\ [:v:]_{\kappa^P_{\iota_2}} \Rightarrow (\iota_2,{*h},[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}}) :: \mathsf{H'}\ ,\ (\iota_2,h,[:v:]_{\kappa^P_{\iota_2}}) :: \mathsf{D'} \\
\qquad\qquad \mid\ \bot \Rightarrow (\iota_2,{*h},[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}}) :: \mathsf{H'}\ ,\ \mathsf{D'} \\
\quad \mid\ (\iota_2,h,[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}}) :: \mathsf{H}_1 \Rightarrow \\
\qquad \mathsf{let\ H'},\mathsf{D'} = \mathsf{dispatch}(\mathsf{CA},\Sigma,\Gamma_{\iota_1},\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\mathsf{H}_1) \\
\qquad \mathsf{in} \\
\qquad\quad \mathsf{match} \\
\qquad\quad\ \mathsf{CA.publish}\left(\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\Gamma_{\iota_1},[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}},\iota_2,\Sigma(\iota_2)\right) \\
\qquad\quad \mathsf{with} \\
\qquad\qquad \mid\ [:v:]_{\kappa^P_{\iota_2}} \Rightarrow \mathsf{H'}\ ,\ (\iota_2,h,[:v:]_{\kappa^P_{\iota_2}}) :: \mathsf{D'} \\
\qquad\qquad \mid\ \bot \Rightarrow (\iota_2,h,[:\tau,\pi_2:]_{\kappa^P_{\mathsf{CA}}}) :: \mathsf{H'}\ ,\ \mathsf{D'} \\
\quad \mathsf{end}
\end{array}
$$

Figure 9: Function dispatch.

When a subscription request is received, the broker stores the corresponding handler in H, maintaining the associated callback address and predicate, and distinguishing between persistent and one-shot entries[1].

Symmetrically, when a publication message is received, the broker checks with certification authority all the subscriptions that it:

$$\frac{\mathsf{dispatch}(\mathsf{CA},\Sigma,\Sigma(\iota_1),\{:[:v,\pi_1:]_{\kappa^P_{\mathsf{CA}},\iota_1:}\}_{\kappa^S_{\iota_1}},\mathsf{H}) = \mathsf{H'},\mathsf{D'}}{\mathsf{CA} \triangleright [\Sigma,\mathsf{H},[\,]] \xrightarrow{\mathsf{pub}(\{:[:v,\pi:]_{\kappa^P_{\mathsf{CA}},\iota:}\}_{\kappa^S_\iota})} \mathsf{CA} \triangleright [\Sigma,\mathsf{H'},\mathsf{D'}]}$$

This rule, which relies on the function dispatch defined in Figure 9, can be applied only when there are no pending messages that should be deliberated at subscribers. The dispatch function is used to iterate over all the active subscriptions to find the ones interested in the published message. When one is found, the corresponding message is added to the dispatching queue. All the matching subscriptions that are not persistent are removed from the list H.

When a set message $\mathsf{set}(\{:[:\tilde{\mathsf{a}},\tilde{\mathsf{v}}:]_{\kappa^P_{\mathsf{CA}},\iota:}\}_{\kappa^S_\iota})$ is received, the broker evolve by updating the attributes environment associated to identifier ι:

$$\frac{\mathsf{CA.set}\left([:\{:\tilde{\mathsf{a}},\tilde{v},\iota:\}_{\kappa^S_\iota}:]_{\kappa^P_{\mathsf{CA}}}\right) = [:\tilde{\mathsf{a}}:]_{\kappa^P_{\mathsf{CA}}},[:\tilde{v}:]_{\kappa^P_{\mathsf{CA}}} \quad \Sigma(\iota) = \Gamma}{\mathsf{CA} \triangleright [\Sigma,\mathsf{H},\mathsf{D}] \xrightarrow{\mathsf{set}(\{:[:\tilde{\mathsf{a}},\tilde{\mathsf{v}}:]_{\kappa^P_{\mathsf{CA}},\iota:}\}_{\kappa^S_\iota})} \mathsf{CA} \triangleright [\Sigma[\iota \leftarrow \Gamma[[:\tilde{\mathsf{a}}:]_{\kappa^P_{\mathsf{CA}}} \leftarrow [:\tilde{v}:]_{\kappa^P_{\mathsf{CA}}}]],\mathsf{H},\mathsf{D}]}$$

[1]Here we use the standard list notation: [] indicates the empty list, while $v :: lst$ denotes the list starting with l and having lst as tail.

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash act \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{A}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash act;\mathsf{A}} \qquad \frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash v : \tau \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \pi : \mathsf{bool}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{pub}(v,\pi)}$$

$$\frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \pi : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}}[x \leftarrow \tau] \vdash \mathsf{Q}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{sub}(\tau,\pi,\lambda x.\mathsf{Q})} \qquad \frac{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \pi : \mathsf{bool} \quad \Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}}[x \leftarrow \tau] \vdash \mathsf{Q}}{\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathsf{sub}(\tau,\pi,\lambda x.\mathsf{Q})}$$

Figure 10: Typing rules for pub-sub processes.

Finally, the following last rule states that when the D is not empty, a broker sends messages to the handler:

$$\mathsf{CA} \triangleright [\Sigma,\mathsf{H},(\iota_2,h,[:v:]_{\kappa^P_{\iota_2}}) :: \mathsf{D}] \xrightarrow{h@\iota\left([:v:]_{\kappa^P_{\iota_2}}\right)} \mathsf{CA} \triangleright [\Sigma,\mathsf{H},\mathsf{D}]$$

In what follows, we let μ denote either the messages that can be received by a broker and the messages sent by the broker.

Components and Processes. Publishers and subscribers are running over *components* of the form $[\Gamma \triangleright \mathsf{A}]_\iota$ where ι is an identity, Γ is an attribute store and A is a process generated by the following syntax:

$$\begin{array}{rcl} \mathsf{A} & ::= & \mathsf{skip} \mid act;\mathsf{A} \mid \mathsf{if\ exp\ then\ A\ else\ Q} \mid h(x).\mathsf{A} \mid *h(x).\mathsf{A} \\ & \mid & \mathsf{when(exp)A} \mid \mathsf{M}[e] \mid \mathsf{A} \mid \mathsf{A} \\ act & ::= & \mathsf{pub}(v,\pi) \mid \mathsf{sub}(\tau,\pi,\lambda x.\mathsf{Q}) \mid \mathsf{sub}^*(\tau,\pi,\lambda \mathsf{x}.\mathsf{Q}) \mid \mathsf{set}[\tilde{\mathsf{a}} \leftarrow \tilde{e}] \end{array}$$

The syntax of pub/sub processes is similar to the one already introduced in Section 2. However, while a CRABC process performs high-level attribute-based input-output operations, a process A executes pub-sub actions that allow to publish a message ($\mathsf{pub}(v,\pi)$), subscribe for a single value ($\mathsf{sub}(\tau,\pi,\lambda x.\mathsf{Q})$) or persistently ($\mathsf{sub}^*(\tau,\pi,\lambda \mathsf{x}.\mathsf{Q})$), and to set attribute values ($\mathsf{set}[\tilde{\mathsf{a}} \leftarrow \tilde{v}]$). Moreover, $h(x).\mathsf{A}$ and $*h(x).\mathsf{A}$ are used to represent a handler waiting for a published value. These constructs cannot be used in a process specification and are generated by the operational semantics that we will introduce in a while. We let ∇ denote the set of process definitions of form $\mathsf{N}(x:\tau) \triangleq \mathsf{A}$. Typing rules of Figure 5 can be extended to manage pub-sub processes by considering the rules of Figure 10.

The operational semantics of components is reported in Figure 11 and follows a standard schema in process algebras. When a process executes an action, the corresponding message is generated. Rules for *if-then-else*, *when* and *call* constructs are omitted since them are the same as the corresponding ones of Figure 7.

System level operational semantics. From a global perspective, a publish-subscribe system consists of a certification authority and a broker that coordinates k different components. This can be described by a term of the form:

$$\mathsf{CA} \triangleright \mathsf{B} \vdash [\Gamma_1 \triangleright \mathsf{A}_1]_{\iota_1} \parallel \cdots \parallel [\Gamma_k \triangleright \mathsf{A}_k]_{\iota_k}$$

The operational semantics of this term is defined in terms of the following single reduction rule:

$$[\Gamma \triangleright \mathsf{pub}(v,\pi);\mathsf{A}]_\iota \xrightarrow{\mathsf{pub}(\{:[:v,\pi\Downarrow^\iota_\Gamma:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota})} [\Gamma \triangleright \mathsf{A}]_\iota \quad \textsc{Pub}$$

$$[\Gamma \triangleright \mathsf{sub}\,(\tau,\pi,\lambda x.\,\mathsf{Q})\ ;\ \mathsf{A}]_\iota \xrightarrow{\mathsf{sub}(\{:[:\tau,\pi\Downarrow^\iota_\Gamma:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})} [\Gamma \triangleright \mathsf{A} \mid h(x).\mathsf{Q}]_\iota \quad \textsc{Sub}$$

$$[\Gamma \triangleright \mathsf{sub}*(\tau)\pi\lambda x.\,\mathsf{Q}\ ;\ \mathsf{A}]_\iota \xrightarrow{\mathsf{sub}*(\{:[:\tau,\pi\Downarrow^\iota_\Gamma:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota},\mathsf{h})} [\Gamma \triangleright \mathsf{A} \mid *h(x).\mathsf{Q}]_\iota \quad \textsc{Sub}*$$

$$\frac{[\![\tilde{e}]\!]^\iota_\Gamma = \tilde{v}}{[\Gamma \triangleright \mathsf{set}[\tilde{\mathsf{a}} \leftarrow \tilde{e}];\ \mathsf{A}]_\iota \xrightarrow{\mathsf{set}(\{:[:\tilde{\mathsf{a}},\tilde{\mathsf{v}}:]_{\kappa^P_{\mathsf{CA}}},\iota:\}_{\kappa^S_\iota})} [\Gamma[\tilde{\mathsf{a}} \leftarrow \tilde{v}] \triangleright \mathsf{A}]_\iota} \ \textsc{Set}$$

$$[\Gamma \triangleright h(x).\mathsf{A}]_\iota \xrightarrow{h@i(v)} [\Gamma \triangleright \mathsf{A}[x \leftarrow v]]_\iota \quad \textsc{Rcv}$$

$$[\Gamma \triangleright *h(x).\mathsf{A}]_\iota \xrightarrow{h@i(v)} [\Gamma \triangleright *h(x).\mathsf{A} \mid \mathsf{A}[x \leftarrow v]]_\iota \quad \textsc{Rcv}*$$

$$\frac{[\Gamma_1 \triangleright \mathsf{A}_1]_\iota \overset{\mu}{\Longrightarrow} [\Gamma_2 \triangleright \mathsf{A}_2]_\iota \quad \mathsf{bn}(\mu) \notin \mathsf{A}}{[\Gamma_1 \triangleright \mathsf{A}_1 \mid \mathsf{A}]_\iota \overset{\mu}{\Longrightarrow} [\Gamma_2 \triangleright \mathsf{A}_2 \mid \mathsf{A}]_\iota} \ \textsc{ParL} \qquad \frac{[\Gamma_1 \triangleright \mathsf{A}_1]_\iota \overset{\mu}{\Longrightarrow} [\Gamma_2 \triangleright \mathsf{A}_2]_\iota \quad \mathsf{bn}(\mu) \notin \mathsf{A}}{[\Gamma_1 \triangleright \mathsf{A} \mid \mathsf{A}_1]_\iota \overset{\mu}{\Longrightarrow} [\Gamma_2 \triangleright \mathsf{A} \mid \mathsf{A}_2]_\iota} \ \textsc{ParR}$$

Figure 11: Operational Semantics of Publish-Subscribe Components.

$$\frac{\mathsf{CA} \triangleright \mathsf{B} \overset{\mu}{\Longrightarrow} \mathsf{CA} \triangleright \mathsf{B}' \quad [\Gamma_i \triangleright \mathsf{A}_i]_{\iota_i} \overset{\mu}{\Longrightarrow} [\Gamma'_i \triangleright \mathsf{A}'_i]_{\iota_i}}{\mathsf{CA} \triangleright \mathsf{B} \vdash \cdots \parallel [\Gamma_i \triangleright \mathsf{A}_i]_{\iota_i} \parallel \cdots \Rightarrow \mathsf{CA} \triangleright \mathsf{B} \vdash \cdots \parallel [\Gamma'_i \triangleright \mathsf{A}'_i]_{\iota_i} \parallel \cdots}$$

4 Implementing CRABC with Secure Publish-Subscribe

In this section, we will show how CRABC can be implemented via Secure Publish-Subscribe. First, we introduce a translation function that maps CRABC processes. Function $\mathscr{T}[\![P]\!]$ is inductively defined as follows:

$$\mathscr{T}[\![\mathsf{skip}]\!] = \mathsf{skip} \qquad \mathscr{T}[\![\langle e\rangle@\pi;P]\!] = \mathsf{pub}(v,\pi)\,\mathbf{A};\mathscr{T}[\![P]\!]$$

$$\mathscr{T}[\![\pi(\mathsf{x}:\tau) \Rightarrow P]\!] = \mathsf{sub}\,(\tau,\pi,\lambda x.\,\mathscr{T}[\![P]\!])\ ;\mathsf{skip}$$

$$\mathscr{T}[\![\mathsf{if}\ (e)\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!] = \mathsf{if}\ e\ \mathsf{then}\ \mathscr{T}[\![P]\!]\ \mathsf{else}\ \mathscr{T}[\![Q]\!]$$

$$\mathscr{T}[\![P \mid Q]\!] = \mathscr{T}[\![P]\!] \mid \mathscr{T}[\![Q]\!] \qquad \mathscr{T}[\![[\tilde{a} \leftarrow \tilde{e}];P]\!] = \mathsf{set}[\tilde{\mathsf{a}} \leftarrow \tilde{v}];\mathscr{T}[\![P]\!]$$

$$\mathscr{T}[\![\mathsf{when}(e)\ P]\!] = \mathsf{when}(e)\mathscr{T}[\![P]\!]$$

Given a set of Δ of CRABC process definitions, we let ∇_Δ be the translated publish-subscribe processes $\mathsf{N}(x:\tau) \triangleq \mathscr{T}[\![P]\!]$ for any $\mathsf{N}(x:\tau) \triangleq P \in \Delta$.

The following lemma guarantees that if P is a well-typed CRABC process, then its translation is a well-typed publish-subscribe process.

Lemma 5 *For any $\Lambda_{\mathbb{A}}$ and $\Lambda_{\mathbb{V}}$, and for any* CRABC *process P, $\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash P$ if and only if $\Lambda_{\mathbb{A}},\Lambda_{\mathbb{V}} \vdash \mathscr{T}[\![P]\!]$.*

Given a CRABC component $[\Gamma \triangleright P]_\iota$ its publish-subscribe version is obtained by considering $[\Gamma \triangleright \mathscr{T}[\![P]\!]]_\iota$. Given a system specification

$$\Lambda_{\mathbb{A}}, \Delta \triangleright [\Gamma_1 \triangleright P_1]_{\iota_1} \parallel \cdots \parallel [\Gamma_n \triangleright P_n]_{\iota_n}$$

its implementation in secure publish-subscribe is defined as:

$$\mathsf{CA} \triangleright [\Sigma_{\mathscr{T}}, [\,], [\,]] \vdash [\Gamma_1 \triangleright \mathscr{T}[\![P_1]\!]]_{\iota_1} \parallel \cdots \parallel [\Gamma_n \triangleright \mathscr{T}[\![P_n]\!]]_{\iota_n}$$

where for each ι_i and attributes a, $\Sigma_{\mathscr{T}}(\iota)([\!: \mathsf{a} :\!]_{\kappa^P_{\mathsf{CA}}}) = [\!: \Gamma_i(\mathsf{a}) :\!]_{\kappa^P_{\mathsf{CA}}}$.

It is easy to see that any computation in the CRABC configuration can be mimicked by the corresponding publish-subscribe implementation. Moreover, any computation in the publish-subscribe implementation will always reach a point that corresponds to a computation in the CRABC specification.

5 Conclusions

In this paper, we have introduced CRABC (Cryptographic Attribute-based Calculus), an extension of ABC that integrates cryptographic primitives. We have seen how the proposed variant integrates *cryptographic primitives* that guarantee data confidentiality and integrity, as well as authentication and non-repudiation.

We have also presented a variant of the classical publish-subscribe paradigm [11] that integrates attribute-based mechanisms together with a secure infrastructure. Finally, we have seen how the proposed framework can be used to implement an execution environment for CRABC.

The proposed approach is somehow related to Attribute-Based Encryption (ABE) [8, 13]. This is a cryptographic primitive in which decryption capabilities are determined by attributes and access policies, rather than by explicit identities. In particular, Ciphertext-Policy ABE (CP-ABE) [21] allows a sender to encrypt data under a policy such that only users whose attributes satisfy the policy can decrypt it. While ABE shares conceptual similarities with attribute-based communication, it operates purely at the cryptographic level and does not provide a process-calculus or coordination semantics. Our approach differs in that attributes govern communication partners and interaction structure within a formal calculus, and cryptographic primitives are integrated to protect exchanged data. In this sense, CRABC complements ABE: whereas ABE enforces access control at decryption time, our model embeds attribute-based selection into both behavioural semantics and secure middleware execution. Exploring a tighter integration between CRABC and ABE-based enforcement mechanisms represents an interesting direction for future work.

As future work, we also plan to use the proposed framework to provide a formal foundation for performing analyses of trustworthiness and threat models. Moreover, we plan to implement the proposed framework and use it in different application scenarios like, for instance, IoT.

References

[1] Alrahman, Y.A., De Nicola, R., Loreti, M.: On the power of attribute-based communication. In: Proc. of the International Conference on Formal Techniques for Distributed Objects, Components, and Systems (FORTE). pp. 1–18 (2016)

[2] Alrahman, Y.A., De Nicola, R., Loreti, M.: A behavioural theory for interactions in collective-adaptive systems. CoRR **abs/1711.09762** (2017), `http://arxiv.org/abs/1711.09762`

[3] Alrahman, Y.A., Garbi, G.: A distributed API for coordinating abc programs. Int. J. Softw. Tools Technol. Transf. **22**(4), 477–496 (2020). https://doi.org/10.1007/S10009-020-00553-4, https://doi.org/10.1007/s10009-020-00553-4

[4] Alrahman, Y.A., Nicola, R.D., Garbi, G., Loreti, M.: A distributed coordination infrastructure for attribute-based interaction. In: Baier, C., Caires, L. (eds.) Formal Techniques for Distributed Objects, Components, and Systems - 38th IFIP WG 6.1 International Conference, FORTE 2018, Held as Part of the 13th International Federated Conference on Distributed Computing Techniques, DisCoTec 2018, Madrid, Spain, June 18-21, 2018, Proceedings. Lecture Notes in Computer Science, vol. 10854, pp. 1–20. Springer (2018). https://doi.org/10.1007/978-3-319-92612-4_1, https://doi.org/10.1007/978-3-319-92612-4_1

[5] Alrahman, Y.A., Nicola, R.D., Loreti, M.: A calculus for collective-adaptive systems and its behavioural theory. Inf. Comput. **268** (2019). https://doi.org/10.1016/J.IC.2019.104457, https://doi.org/10.1016/j.ic.2019.104457

[6] Alrahman, Y.A., Nicola, R.D., Loreti, M.: Programming interactions in collective adaptive systems by relying on attribute-based communication. Sci. Comput. Program. **192**, 102428 (2020). https://doi.org/10.1016/J.SCICO.2020.102428, https://doi.org/10.1016/j.scico.2020.102428

[7] Anderson, S., Bredeche, N., Eiben, A., Kampis, G., van Steen, M.: Adaptive collective systems: herding black sheep. BookSprints for ICT Research (2013)

[8] Bethencourt, J., Sahai, A., Waters, B.: Ciphertext-policy attribute-based encryption. In: Proceedings - IEEE Symposium on Security and Privacy. p. 321 – 334 (2007). https://doi.org/10.1109/SP.2007.11, https://www.scopus.com/inward/record.uri?eid=2-s2.0-34548731375&doi=10.1109%2fSP.2007.11&partnerID=40&md5=5f7ac9261e2e7f4ab82e6b40c7e66f21

[9] Comini, M., Gemolotto, L., Miculan, M.: Attribute-based communication over pub/sub: Transactional coordination for smart systems. In: Ferreira, C., Mezzina, C.A. (eds.) Formal Techniques for Distributed Objects, Components, and Systems - 45th IFIP WG 6.1 International Conference, FORTE 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025, Lille, France, June 16-20, 2025, Proceedings. Lecture Notes in Computer Science, vol. 15732, pp. 96–113. Springer (2025). https://doi.org/10.1007/978-3-031-95497-9_6, https://doi.org/10.1007/978-3-031-95497-9_6

[10] De Nicola, R., Loreti, M., Pugliese, R., Tiezzi, F.: A formal approach to autonomic systems programming: The SCEL language. ACM Transactions on Autonomous and Adaptive Systems (TAAS) **9**(2), 7:1–7:29 (2014)

[11] Eugster, P.T., Felber, P., Guerraoui, R., Kermarrec, A.: The many faces of publish/subscribe. ACM Comput. Surv. **35**(2), 114–131 (2003). https://doi.org/10.1145/857076.857078, https://doi.org/10.1145/857076.857078

[12] Ferscha, A.: Collective adaptive systems. In: Proc. of the International Symposium on Wearable Computers (ISWC). pp. 893–895 (2015)

[13] Goyal, V., Pandey, O., Sahai, A., Waters, B.: Attribute-based encryption for fine-grained access control of encrypted data. In: Proceedings of the ACM Conference on Computer and Communications Security. p. 89 – 98 (2006). https://doi.org/10.1145/1180405.1180418, https://www.scopus.com/inward/record.uri?eid=2-s2.0-34547273527&doi=10.1145%2f1180405.1180418&partnerID=40&md5=e977fb207a5e15bbcb4d7dcaba7f4857, cited by: 4649

[14] Menezes, A.J., Vanstone, S.A., Oorschot, P.C.V.: Handbook of Applied Cryptography. CRC Press, Inc., USA, 1st edn. (1996)

[15] Nicola, R.D., Duong, T., Loreti, M.: ABEL - A domain specific framework for programming with attribute-based communication. In: Nielson, H.R., Tuosto, E. (eds.) Coordination Models and Languages - 21st IFIP WG 6.1 International Conference, COORDINATION 2019, Held as Part of the 14th International Federated Conference on Distributed Computing Techniques, DisCoTec 2019, Kongens Lyngby, Denmark, June 17-21, 2019, Proceedings. Lecture Notes in Computer Science, vol. 11533, pp. 111–128. Springer (2019). https://doi.org/10.1007/978-3-030-22397-7_7, https://doi.org/10.1007/978-3-030-22397-7_7

[16] Nicola, R.D., Duong, T., Loreti, M.: Provably correct implementation of the *AbC* calculus. Sci. Comput. Program. **202**, 102567 (2021). https://doi.org/10.1016/J.SCICO.2020.102567, https://doi.org/10.1016/j.scico.2020.102567

[17] Pasqua, M., Miculan, M.: Abu: A calculus for distributed event-driven programming with attribute-based interaction. Theor. Comput. Sci. **958**, 113841 (2023). https://doi.org/10.1016/J.TCS.2023.113841, https://doi.org/10.1016/j.tcs.2023.113841

[18] Pasqua, M., Miculan, M.: Behavioral equivalences for abu: Verifying security and safety in distributed iot systems. Theor. Comput. Sci. **998**, 114537 (2024). https://doi.org/10.1016/J.TCS.2024.114537, https://doi.org/10.1016/j.tcs.2024.114537

[19] Prasad, K.V.: A calculus of broadcasting systems. Science of Computer Programming **25**(2-3), 285–327 (1995)

[20] Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM **21**(2), 120–126 (1978)

[21] Waters, B.: Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) **6571 LNCS**, 53 – 70 (2011). https://doi.org/10.1007/978-3-642-19379-8_4, https://www.scopus.com/inward/record.uri?eid=2-s2.0-79952521560&doi=10.1007%2f978-3-642-19379-8_4&partnerID=40&md5=6d294bc2aadb5cb1635e3e90d35d1698